Final Report: Mentre Study Assistant

James Bommentre and Leonard Chiang
ISGB 79AM - RAG and Context Engineering
Dr. Apostolos Filippas
Wednesday 06 May 2026

Disclosure: an early draft of this report was generated using artificial intelligence,
but this version has been extensively rewritten;
a version history is available on an as-needed basis
to enable due diligence compare-and-contrast.

Motivation & Use Cases **2**
Methodology **2**
Conclusions and Takeaways **4**
Limitations & Future Directions **4**

Motivation & Use Cases

Text-based academic materials that students and practitioners encounter can at times seem daunting in their protracted length, deep level of detail, and complicated, technical phrasing. Throughout the centuries, summarization has remained a technique for improving retention of such dense material; in recent years, the development and widespread adoption of large language models (LLMs) has enabled and driven the rapid processing of natural language sources into succinct summaries (a workflow surely already familiar to countless learners). We combine this text processing ability of artificial intelligence (AI) with its ability to navigate programming syntax to further concentrate textual summary into visual renderings of key concepts through Mermaid, a tool that uses Markdown-inspired syntax to automate the creation of diagrams, e.g. flowcharts. The parallel presentations– text and image– conveying the same knowledge and different ways, in combination with a co-author's surname, yield the name Mentre: Italian for "while."

The speed afforded by AI is of particular utility in a fast-paced modern world, wherein students under pressure to maintain academic performance amid a bevy of undergraduate admissions-relevant extracurriculars and a desire to maintain a healthy social life may find traditional techniques of deploying a half-dozen colored markers on a big five-subject spiral-bound notebook infeasible. Mentre, a web-based, hence hand-held, study tool can be used even in space-constrained settings, such as commutes on the city subway system. Even those more advanced in their academic careers can find value in accelerating progress through their research pipeline; academic articles can be condensed down into diagrams that would fit on, say, index cards, rather than printouts that could run into ream-scale volumes of paper.

Methodology

Mentre's operation is extremely lightweight, spanning less than 300 lines in a Python file. Python's extensive community support, however, means that this single file can handle both the front- and back-ends, through Gradio plus some vibe-coded HTML, and abstracted LLM access through LiteLLM, respectively.

A pillar of this project's associated course is retrieval-augmented generation (RAG): that is, a pre-trained general-knowledge system is given a query that supplies additional contextualizing information hailing from outside its training corpus, and the ingestion thereof brings the respective "knowledge bases" of user and system into improved alignment, such that the system's generated output does not stray from the user's focus. In Mentre's case, the retrieval is not search-mediated but directly supplied by the user.

Users have multiple options by which to supply this material.

- A rudimentary paste-in of text.
    - This material can be ingested immediately as a string data-type.

- A file upload in the commonly-encountered plaintext (.txt) or Portable Document Formats (.pdf).
    - This requires file handling; cf. function extract_text_from_upload(path) (starting line 63) in the enclosed Python (.py) file. The invocation of the pypdf library, including the error handling of inaccessible PDFs, comes at the behest of Cursor Agent, illustrating how generative AI copilots can contribute to programming outcomes.
- The inputting of a URL.
    - A scraper built on beautifulsoup4 (bs4) attempts to extract body text from the provided webpage, culling other graphical and navigational elements.
        - (Failing this, the user may consider copy-pasting the body text.)

Large language models are by nature stateless stochastic systems, i.e. their outputs may vary with each passing of a given input. Their outputs may also be quite verbose or even prevaricating. Students, meanwhile, might be responding to relatively closed-form assignments requesting short, precise answers. To reconcile this difference, we in our first step introduce a Pydantic BaseModel schema with four Fields: title, summary, key_concepts, and study_questions. We impose this schema on the LLM's output, structuring the result to more closely adhere to the anticipated usage mode.

Stochastic outputs are also in conflict with the constrained degree of syntactical flexibility that programming(-adjacent) content like Mermaid Markdown permits. To address this, we deploy another technique that limits LLM behavior: prompt engineering. This simply comprises a set of straightforward developer-issued instructions for guidance toward desired behavior; these are present as lines 193-199 in the submitted code. Backstopping this is some simple postprocessing (line 211) that removes extra characters that could trip up diagram rendering.

We select OpenAI's gpt-5-nano offering for these tasks largely out of convenience; it is, first and foremost, accessible through LiteLLM, a tool already used extensively in class, while its relatively recent vintage and parameter size represent a reasonable compromise between frontier-model performance and cost- and latency-wiseness.

For its part, Mermaid.ai was the rendering engine of choice initially due to co-author Chiang's familiarity with its syntax and automatic layout rendering through its click-and-type Live Editor. Its offering of programmatic usage presented a learning opportunity for both co-authors. Ultimately, Mermaid diagrams are rendered directly on the same webpage through dynamic HTML, though the underlying syntax is also displayed to the user to copy over to mermaid.live, in case of failure.

The Gradio library supplies the graphical user interface that turns the script into a web application accommodating the abovementioned user actions, including a timer tracking the processing time and a publicly shareable link < https://865dd2ca2aca36f0f4.gradio.live/ >.

Conclusions and Takeaways

Mentre's performance is satisfactory, synthesizing techniques covered in the course into an interactive, public-facing product.

As covered in class, preprocessing the data is a critical step, and this principle is followed through the use of bs4 to strip away extraneous webpage elements to bring exclusive focus to the substance of the study material.

Likewise, this project shows the importance and nuances of managing the variable nature of LLM outputs. Despite prior exhortation to "[r]eturn only Mermaid code" (line 193), earlier versions of Mentre were still using parentheses that would break the diagram rendering, so a further instruction re "avoiding special characters" (198) was needed.

Another aspect of AI engineering covered by this project is cost consideration. While OpenAI has moved onto GPT-5.5, our selection remains dependable and for the designated use case, suggesting that engineers don't always need to chase the "hot new thing" just for the sake of it.

A final takeaway is not to overcommit (no pun intended) to programming "from scratch." A sizable portion of the .py file was derived from coauthor Bommentre's ChatGPT exchanges– not even the more advanced Codex offering– and turned out to be quite serviceable. This notion also extends to using preexisting Python-friendly app development tools like Gradio to decrease the time to market. Nor does building upon others' experiences extend only to faceless folks on the other side of the Internet: the idea of using Gradio at all arose from co-author Chiang's in-person audit-attendance of Fordham Prof. Daniel Groner's Advanced Python class.

Limitations & Future Directions

One concern is that Mentre ingests material under copyright or other intellectual property protections, and according to a conversation that co-author Chiang had with Fordham faculty member Mark Conrad, the creation of derivative diagrams may not be considered a fair use under the laws and customs governing such material. For this reason, the .txt file included in the project submission was written by Chiang directly for the use of testing this study tool, with his authorial familiarity serving as the evaluation metric. The addressing of this copyright problem feels outside the scope of this course, but is worth mentioning as potentially a grave issue in any broader deployment context.

In other cases, Mentre may not be able to ingest any material at all because the PDFs supplied are scans (i.e. images) of printed material. Allowing image uploads was put on holds due to cost and feasibility concerns, but, again, in broader deployment contexts this modality could be of great relevance, especially to the mobile-native generation students very much accustomed to using phone cameras.

Furthermore, Mentre is capped at 100,000 characters (not words) to preserve latency. This limitation might frustrate some users who would have hoped to process longer works, such as epic poems. Chunking and compaction strategies discussed in course lectures are reserved for future work.

On the technical side, Mentre is, notably, tightly corralled by prompt to just top-down flowcharts to reduce the possibility of errors, given that Mermaid syntax varies somewhat across its offered diagram types. Serious scholars, however, will recognize the utility of other diagram types. For instance, interactions between narrative characters could be documented using sequence and swimlane diagrams; even more imaginatively, characters' attitudes and attributes could be visualized as radar charts. Were Mentre eventually to offer this level of flexibility, these options could be user-guided through the use of Gradio option-selection mechanisms, like dropdowns and/or checkboxes.

Even given this restriction, the diagrams currently produced by Mentre can vary fairly markedly across runs. In a generous interpretation, students can re-generate distinct diagrams until they receive one that resonate better with their recall ability; in a worst-case scenario, diagrams may not reliably capture the information with adequate fidelity to the source material.

We must also acknowledge that the large language models are called as-is, without finetuning to align to any particular problem domain. Specialists hoping to translate articles detailing industry-specific innovations might butt up against the general-purpose LLM's knowledge cutoff and lack of subject coverage in its training data. The co-authors, like the Internet at large (and hence said training data), are strongly biased toward English, which may leave blindspots regarding translations and sociolinguistics of other languages.

As a parting note, another limitation of Mentre's status quo is that there does not exist a clearly-defined user tracking. This means that a given user cannot use credentials to access their previously generated images, nor can malicious usage be easily traced. Future instantiations of Mentre would need to incorporate database design and management to create logs of user behavior– for their sake and ours.